SPLITSHARE

Guia Oficial do Grupo e Arquitetura do Projeto

Material de estudo: Gabriela Marotti dos Santos, Lorena de Oliveira Dias e Maria Eduarda de Oliveira Carvalho
Elaborado em: 08/11/2026 (a partir da versão simplificada do projeto)

1. Como o projeto funciona

O **SplitShare** permite cadastrar moradores e dividir despesas de uma moradia. O usuário interage pelo terminal. O Java valida os dados, calcula as parcelas e acessa o MySQL para salvar e recuperar os registros.

> **O caminho de uma operação**
>
> ```
> Usuário > Main -> Console SplitShare
>   +-> EntradaConsole (ler e converter)
>   +-> modelos (validar e calcular)
>   +-> DAOs -> ConexaoJdbc -> MySQL
> ```

Exemplo: Ao escolher cadastrar morador, o menu lê nome, e-mail e celular, cria um objeto `Morador` e chama `MoradorDAO.inserir`. O banco gera um ID. O DAO coloca esse ID no objeto e o menu confirma o cadastro.

Pastas e Responsabilidades

`src/main/java/br/com/splitshare/` guarda o programa. Dentro dela:

- **model:** representa moradores e despesas.
- **console:** cuida do terminal.
- **dao:** concentra as operações SQL.
- **Main.java:** inicia tudo.

`database/` guarda a estrutura das tabelas. `docs/` contém os diagramas. O `README` apresenta requisitos, cronograma e execução. O `pom.xml` define a compilação.

Requisitos na Prática

- **RF01** CRUD de moradores.
- **RF02** Cadastra despesas fixas para todos os moradores atuais.
- **RF03** Seleciona participantes de despesas variáveis.
- **RF04** Lista totais e parcelas.
- **RF05** Edita e exclui despesas.

CRUD significa criar, consultar, atualizar e excluir. Para demonstrar persistência, fechem o programa, abram novamente e consultem um registro já cadastrado.

2. Arquivos Java, um por um

Os caminhos abaixo são relativos a `src/main/java/br/com/splitshare/`.

`Main.java`

Ponto de entrada. Monta a conexão, os dois DAOs e o menu. Sem argumentos, abre o sistema conectado ao banco. `--help` mostra o uso. `--demo` executa exemplos em memória e não grava dados.

`model/Morador.java`

Representa ID, nome, e-mail e celular. Construtores e setters validam os dados. ID zero representa um novo objeto que ainda não recebeu ID do banco.

`model/Despesa.java`

Classe abstrata com ID, descrição, valor total e vencimento. Centraliza validações e o rateio exato dos centavos. Define métodos que as subclasses devem implementar.

`model/DespesaFixa.java & DespesaVariavel.java`

Fixa: Herda Despesa. Recebe todos os moradores do cadastro da despesa e guarda cópia da lista.
Variável: Herda Despesa. Permite adicionar/remover até 15 participantes selecionados. O salvamento exige ao menos um.

`dao/ConexaoJdbc.java`

Lê as variáveis de ambiente, abre conexões JDBC e coordena commit e rollback. Recupera o ID gerado pelo banco.

`dao/MoradorDAO.java & DespesaDAO.java`

MoradorDAO: Executa INSERT, SELECT, UPDATE, DELETE e converte resultados para objetos.
DespesaDAO: Grava despesas e participantes juntos, lista, busca, edita e exclui. Reconstrói os objetos conforme o tipo salvo.

`console/EntradaConsole.java & ConsoleSplitShare.java`

Entrada: Lê linhas, converte IDs, valores, datas e trata cancelamentos.
Console: Mostra opções de 0 a 9, chama DAOs, seleciona moradores, exibe parcelas e trata falhas.

3. Arquivos de Apoio e Banco de Dados

Configurações e Docs: O `pom.xml` usa Java 17, baixa o MySQL Connector/J e usa o plugin Shade para gerar o JAR executável. O `README.md` resume o projeto, ODS e diagramas. O `.gitignore` oculta binários e credenciais.

As Três Tabelas (database/schema.sql)

Deve ser executado no MySQL antes de usar a aplicação.

- `morador`: guarda ID, nome, e-mail único e celular.
- `despesa`: guarda ID, descrição, valor DECIMAL(10,2), vencimento e tipo.
- `despesa_morador`: guarda um par de IDs por participação.

```
PRIMARY KEY (id_despesa, id_morador) ON DELETE CASCADE ON DELETE RESTRICT
```

CASCADE: apaga as participações quando a despesa é excluída.
RESTRICT: impede excluir alguém que participa de despesas.

Credenciais Locais: O programa recebe as credenciais `SPLITSHARE_DB_URL`, `_USER` e `_PASSWORD` pelo ambiente, mantendo os dados fora do código-fonte.

4. POO e Divisão dos Centavos

Herança, abstração e polimorfismo

`Despesa` é abstrata. `DespesaFixa` e `DespesaVariavel` usam `extends Despesa`.

```
@Override public BigDecimal calcularRateio() { return
calcularCotaBase(moradoresEnvolvidos.size()); }
```

Isso é **polimorfismo**: o Java escolhe a implementação do objeto real durante a execução.

Por que usar BigDecimal?

Dinheiro exige precisão decimal (duas casas). `RoundingMode.DOWN` calcula a cota base sem arredondar para cima. Os centavos restantes são rateados um a um.

```
long centavosTotais = valorTotal.movePointRight(2).longValueExact(); long
centavosBase = cotaBase.movePointRight(2).longValueExact(); long restantes =
centavosTotais - (centavosBase * participantes.size());
```

Se R$ 100,00 for dividido entre 3 pessoas, a base é R$ 33,33. Sobra: 10000 - (3333 × 3) = 1 centavo. O laço soma R$ 0,01 à primeira parcela, resultando em: R$ 33,34 + R$ 33,33 + R$ 33,33 = R$ 100,00.

Proteção de dados: `Collections.unmodifiableList` impede que o chamador altere a lista diretamente, e `copiarParticipantes` previne objetos repetidos.

5. JDBC, Consultas e Transações

```
String sql = "DELETE FROM morador WHERE id_morador = ?"; try (Connection conexao =
banco.abrir(); PreparedStatement comando = conexao.prepareStatement(sql)) {
comando.setInt(1, id); return comando.executeUpdate() == 1; }
```

Os pontos de interrogação (`?`) recebem os valores de forma segura (consultas parametrizadas). O bloco `try-with-resources` fecha tudo automaticamente.

Gravação de Despesa e Participantes (Transações)

O método `emTransacao` desativa o commit automático e só salva se tudo der certo:

```
conexao.setAutoCommit(false); try { T resultado = operacao.executar(conexao);
conexao.commit(); return resultado; } catch (SQLException | RuntimeException falha)
{ conexao.rollback(); throw falha; }
```

Se a inserção do vínculo falhar, o `rollback()` desfaz a transação, evitando que a despesa fique órfã no banco.

6. Entrada, Execução e Apresentação

Trechos de Código Comuns

- `Optional` e `orElseThrow`: Lida com resultados vazios ao buscar por ID.
- `split(",", -1)`: Divide as entradas do menu, rejeitando vírgulas sobrando.
- **Datas Seguras:** O formato `ResolverStyle.STRICT` rejeita datas como 31/02/2026.
- **Expressões Regulares:** Aceita valores positivos formatados, trocando vírgulas por pontos para o `BigDecimal`.

> **Como iniciar o projeto**
> No terminal, com as variáveis de banco já configuradas:
>
> ```
> mvn clean package java -jar target/splitshare.jar
> ```

Roteiro de Estudo em Grupo

1. Cadastrem três moradores e anotem os IDs.
2. Cadastrem uma despesa fixa de R$ 100,00 e expliquem os centavos.
3. Cadastrem uma variável de R$ 150,00 para dois moradores.
4. Fechem e reabram para comprovar a persistência.
5. Editem a variável e expliquem a troca dos vínculos.
6. Observem o bloqueio ao excluir um morador vinculado.

Objetivo: Cada integrante deve conseguir explicar o caminho entre menu, modelo, DAO e banco, apontar um @Override e explicar por que o rollback é necessário.